

Archivos .cpp (proyecto final)

bandit.cpp

```
#include "bandit.h"

#include <cmath> // Para calcular distancia (hipotenusa)

Bandit::Bandit(Entity *target)
{
    targetPlayer = target;
    setPixmap(QPixmap(":/sprites/bandit_agent.png").scaled(70, 70, Qt::KeepAspectRatio));
    speed = 120.0; // Un poco más lento que el jugador para que se pueda escapar
}

void Bandit::update(double dt)
{
    if (!targetPlayer) return;

    // INTELIGENCIA ARTIFICIAL (Persecución Básica)
    // 1. Calcular vector hacia el jugador
    double dx = targetPlayer->x() - x();
    double dy = targetPlayer->y() - y();

    // 2. Calcular distancia (pitágoras)
    double distance = std::sqrt(dx*dx + dy*dy);

    // 3. Normalizar y mover (si está lejos)
    if (distance > 5.0) {
        vx = (dx / distance) * speed;
        vy = (dy / distance) * speed;
```

```

    } else {
        vx = 0; vy = 0;
    }

    Entity::update(dt);
}

```

entity.cpp

```

#include "entity.h"

```

```

Entity::Entity(QGraphicsItem *parent) : QGraphicsPixmapItem(parent)
{
    vx = 0;
    vy = 0;
}

```

```

void Entity::update(double dt)
{
    // Fórmula física básica: Posición = Posición + (Velocidad * tiempo)
    // El dt viene en segundos (ej: 0.016 para 60 FPS)
    setPos(x() + vx * dt, y() + vy * dt);
}

```

```

void Entity::setVelocity(double dx, double dy)
{
    vx = dx;
    vy = dy;
}

```

game.cpp

```
#include "game.h"
#include "level1.h"
#include "level2.h"
#include "level3.h" // [IMPORTANTE] Necesario para conocer el Nivel 3
#include <QDebug>
```

```
Game::Game(QObject *parent) : QObject(parent)
{
    // Crear la escena donde ocurre todo
    scene = new QGraphicsScene(this);
    scene->setSceneRect(0, 0, 1280, 720);

    // Configurar el reloj del juego
    timer = new QTimer(this);
    connect(timer, &QTimer::timeout, this, &Game::gameLoop);

    currentLevel = nullptr;
}
```

```
void Game::start()
{
    // Iniciar directamente en el Nivel 1
    switchLevel(1);

    // Arrancar el bucle a aprox 60 FPS (16ms)
    timer->start(16);
}
```

```
void Game::switchLevel(int levelId)
{
    // 1. Limpiar el nivel actual si existe
    if (currentLevel) {
        currentLevel->clear();
        delete currentLevel;
        currentLevel = nullptr;
    }

    // 2. Seleccionar el nuevo nivel
    if (levelId == 1) {
        currentLevel = new Level1(scene, this);
    }
    else if (levelId == 2) {
        currentLevel = new Level2(scene, this);
    }
    else if (levelId == 3) {
        // [NUEVO] Instanciamos el Nivel de la Luna
        currentLevel = new Level3(scene, this);
    }

    // 3. Inicializar el nivel cargado
    if (currentLevel) {
        currentLevel->init();
    }
}
```

```

void Game::gameLoop()
{
    // Delta time fijo para físicas estables
    double dt = 0.016;

    if (currentLevel) {
        currentLevel->update(dt);
    }

    // Renderizar cambios en pantalla
    scene->advance();
}

```

gamewindow.cpp

```

#include "gamewindow.h"
#include "game.h"    // Necesario para crear el objeto 'game'
#include <QBrush>     // Necesario para usar colores de fondo

GameWindow::GameWindow(QWidget *parent)
    : QMainWindow(parent)
{
    // 1. Configurar la ventana principal
    this->setFixedSize(1280, 720);
    this->setWindowTitle("Cronicas del Tiempo - Debug");

    // 2. Inicializar el controlador del juego
    game = new Game(this);
}

```

```

// 3. Configurar el visor (View)

// Usamos la escena que está dentro de 'game'
view = new QGraphicsView(game->getScene(), this);
view->setFixedSize(1280, 720);

// PRUEBA: Fondo gris oscuro para confirmar que el view carga
view->setBackgroundBrush(QBrush(Qt::darkGray));

// Quitar barras de desplazamiento
view->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
view->setVerticalScrollBarPolicy(Qt::ScrollBarAlwaysOff);

// Poner el view como el widget central de la ventana
setCentralWidget(view);

// 4. Iniciar el juego
game->start();
}

```

```

GameWindow::~GameWindow()
{
}

```

lander.cpp

```

#include "lander.h"

#include <QtMath> // Para qSin, qCos, qSqrt
#include <QDebug>

```

```
Lander::Lander()
{
    // --- 1. CONFIGURAR LA NAVE ---

    QPixmap sprite(":/sprites/lander_module.png");
    setPixmap(sprite.scaled(60, 60, Qt::KeepAspectRatio));

    // Definir el centro de rotación (mitad de 60x60)
    setTransformOriginPoint(30, 30);

    // --- 2. CONFIGURAR LA LLAMA (HIJO DE LA NAVE) ---
    // Cargamos la llama
    QPixmap flameSprite(":/sprites/thruster_flame.png");
    flame = new QGraphicsPixmapItem(flameSprite.scaled(30, 40, Qt::KeepAspectRatio));

    // ¡TRUCO!: Hacemos que la llama sea "hija" de la nave.
    // Así, si rotamos la nave, la llama rota con ella.
    flame->setParentItem(this);

    // Posicionamos la llama debajo de la nave (ajusta el 50 según se vea mejor)
    // X = 15 para centrarla (la nave mide 60, la llama 30 -> sobra 30 -> mitad 15)
    flame->setPos(15, 55);

    // La ocultamos al inicio
    flame->setVisible(false);

    // --- 3. FÍSICAS ---
    vx = 0;
    vy = 0;
```

```

rotationSpeed = 150.0;

thrustPower = 350.0; // Un poco más de potencia para compensar gravedad

gravity = 60.0; // Gravedad lunar

fuel = 100.0;

// NUEVO: Velocidad máxima (Terminal Velocity)
maxSpeed = 200.0; // Límite para que no sea incontrolable

rotatingLeft = false;
rotatingRight = false;
thrusting = false;

setFlag(QGraphicsItem::ItemIsFocusable);
}

void Lander::update(double dt)
{
    // --- 1. ROTACIÓN ---
    if (rotatingLeft) {
        setRotation(rotation() - rotationSpeed * dt);
    }
    if (rotatingRight) {
        setRotation(rotation() + rotationSpeed * dt);
    }

    // --- 2. FÍSICA VECTORIAL ---
    // Aplicar gravedad constante

```



```
vy += gravity * dt;
```

```
// --- 3. PROPULSIÓN Y VISUALIZACIÓN ---
```

```
if (thrusting && fuel > 0) {
```

```
    // Mostrar la llama
```

```
    flame->setVisible(true);
```

```
    fuel -= 15.0 * dt; // Consumo de combustible
```

```
    // Calcular ángulo (restamos 90 porque 0 grados es derecha)
```

```
    double angleRad = qDegreesToRadians(rotation() - 90);
```

```
    // Aplicar fuerza
```

```
    double forceX = thrustPower * qCos(angleRad);
```

```
    double forceY = thrustPower * qSin(angleRad);
```

```
    vx += forceX * dt;
```

```
    vy += forceY * dt;
```

```
}
```

```
else {
```

```
    // Ocultar la llama si no empujamos o no hay fuel
```

```
    flame->setVisible(false);
```

```
}
```

```
// --- 4. LÍMITE DE VELOCIDAD (Clamping) ---
```

```
// Calculamos la velocidad total actual (Pitágoras)
```

```
double currentSpeed = qSqrt(vx*vx + vy*vy);
```

```

// Si vamos más rápido que el límite...
if (currentSpeed > maxSpeed) {
    // Normalizamos el vector y lo multiplicamos por el máximo
    // Esto mantiene la dirección pero reduce la velocidad
    vx = (vx / currentSpeed) * maxSpeed;
    vy = (vy / currentSpeed) * maxSpeed;
}

// --- 5. MOVER ---
Entity::update(dt);

// --- 6. LÍMITES DE PANTALLA ---
// Rebote suave en los lados
if (x() < 0) { setPos(0, y()); vx = -vx * 0.5; }
if (x() > 1200) { setPos(1200, y()); vx = -vx * 0.5; }

// Techo
if (y() < 0) { setPos(x(), 0); vy = 0; }
}

void Lander::keyPressEvent(QKeyEvent *event)
{
    if (event->key() == Qt::Key_Left || event->key() == Qt::Key_A) rotatingLeft = true;
    if (event->key() == Qt::Key_Right || event->key() == Qt::Key_D) rotatingRight = true;
    if (event->key() == Qt::Key_Space || event->key() == Qt::Key_Up || event->key() ==
Qt::Key_W) thrusting = true;
}

void Lander::keyReleaseEvent(QKeyEvent *event)

```

```

{
    if (event->key() == Qt::Key_Left || event->key() == Qt::Key_A) rotatingLeft = false;
    if (event->key() == Qt::Key_Right || event->key() == Qt::Key_D) rotatingRight = false;
    if (event->key() == Qt::Key_Space || event->key() == Qt::Key_Up || event->key() ==
Qt::Key_W) thrusting = false;
}

```

level1.cpp

```

#include "level1.h"

#include "game.h" // Importante incluirlo para usar switchLevel
#include <QDebug>

Level1::Level1(QGraphicsScene *scene, Game* game) : LevelBase(scene, game)
{
}

void Level1::init()
{
    scene->clear();

    // 1. Fondo y Suelo
    QPixmap bg(":/sprites/background_maraton.png");
    background = new QGraphicsPixmapItem(bg.scaled(1280, 720,
Qt::IgnoreAspectRatio));
    scene->addItem(background);

    QPixmap groundImg(":/sprites/ground_maraton.png");
    QGraphicsPixmapItem* ground = new QGraphicsPixmapItem(groundImg.scaled(1280,
100));

```

```

ground->setPos(0, 620);
scene->addItem(ground);

// 2. META (Bandera)
QPixmap flagImg(":/sprites/finish_flag_maraton.png");
finishFlag = new QGraphicsPixmapItem(flagImg.scaled(100, 100,
Qt::KeepAspectRatio));
finishFlag->setPos(1100, 520); // Al final de la pantalla
scene->addItem(finishFlag);

// 3. Jugador
player = new PlayerMaraton();
player->setPos(100, 500);
player->setZValue(10);
scene->addItem(player);
player->setFocus();
}

void Level1::update(double dt)
{
    if (player) {
        player->update(dt);

        // DETECTAR VICTORIA
        if (player->collidesWithItem(finishFlag)) {
            qDebug() << "¡Nivel 1 Completado!";
            game->switchLevel(2); // PASAR AL SIGUIENTE NIVEL
        }
    }
}

```

```
}
```

level2.cpp

```
#include "level2.h"
```

```
#include "game.h" // Necesario para llamar a switchLevel
```

```
#include <QDebug>
```

```
#include <QBrush>
```

```
Level2::Level2(QGraphicsScene *scene, Game *game) : LevelBase(scene, game)
```

```
{
```

```
}
```

```
void Level2::init()
```

```
{
```

```
    scene->clear();
```

```
    // 1. FONDO (Arena del desierto)
```

```
    QPixmap bg(":/sprites/terrain_sand.png");
```

```
    // Escalamos la textura para cubrir toda la pantalla HD
```

```
    background = new QGraphicsPixmapItem(bg.scaled(1280, 720));
```

```
    background->setZValue(-1); // Al fondo
```

```
    scene->addItem(background);
```

```
    // 2. META (El Mercado / Trading Post)
```

```
    market = new QGraphicsPixmapItem(QPixmap(":/sprites/trading_post.png").scaled(150, 150, Qt::KeepAspectRatio));
```

```
    market->setPos(1100, 50); // Esquina superior derecha
```

```
    scene->addItem(market);
```

```

// 3. JUGADOR (Caravana)
player = new PlayerCaravan();
player->setPos(50, 600); // Empieza abajo a la izquierda
scene->addItem(player);
player->setFocus(); // Importante para moverlo con teclas

// 4. ENEMIGO (Bandido)
enemy = new Bandit(player); // Le pasamos el jugador para que lo persiga
enemy->setPos(600, 300); // Empieza en el centro
scene->addItem(enemy);
}

void Level2::update(double dt)
{
    if (player) player->update(dt);
    if (enemy) enemy->update(dt);

    // --- COLISIONES ---

    // 1. VICTORIA: Si tocamos el mercado, vamos al Nivel 3
    if (player->collidesWithItem(market)) {
        qDebug() << "¡Nivel 2 Completado! Viajando a la Luna...";
        game->switchLevel(3); // [IMPORTANTE] Cambio de nivel
        return; // Salimos para evitar procesar más lógica en este frame
    }

    // 2. DERROTA: Si el bandido nos toca
    if (player->collidesWithItem(enemy)) {

```

```

        qDebug() << "¡TE ROBARON! - Regresando al inicio";

        // Penalización: Reiniciar posición
        player->setPos(50, 600);
    }
}

```

level3.cpp

```

#include "level3.h"
#include <QDebug>
#include <QBrush>

Level3::Level3(QGraphicsScene *scene, Game *game) : LevelBase(scene, game)
{
}

void Level3::init()
{
    scene->clear();
    levelFinished = false;

    // 1. FONDO (Superficie Lunar)
    QPixmap bg(":/sprites/lunar_surface.png");
    // Como es textura, la estiramos a toda la pantalla
    background = new QGraphicsPixmapItem(bg.scaled(1280, 720));
    background->setZValue(-1);
    scene->addItem(background);

    // 2. ZONA DE ATERRIZAJE (Landing Zone)

```

```

QPixmap padImg(":/sprites/landing_zone.png");
landingZone = new QGraphicsPixmapItem(padImg.scaled(200, 50)); // Plano y ancho
landingZone->setPos(540, 650); // Abajo al centro
scene->addItem(landingZone);

// 3. JUGADOR (Módulo Lunar)
player = new Lander();
player->setPos(100, 100); // Empezamos arriba a la izquierda
scene->addItem(player);
player->setFocus();
}

void Level3::update(double dt)
{
    if (player && !levelFinished) {
        player->update(dt);

        // --- LÓGICA DE ATERRIZAJE ---

        // Verificar colisión con la plataforma
        if (player->collidesWithItem(landingZone)) {

            // Requisito: Aterrizar suave (velocidad baja)
            // Si cae muy rápido (vy > 80), explota
            if (player->getSpeedY() > 80.0) {
                qDebug() << "¡CRASH! Aterrizaje muy rápido.";
                scene->setBackgroundBrush(Qt::red); // Feedback visual de error
                player->setPos(100, 100); // Reiniciar
            }
        }
    }
}

```



```

    }
    else {
        qDebug() << "¡MISIÓN CUMPLIDA! Has conquistado la Luna.";
        scene->setBackgroundBrush(Qt::blue); // Feedback de victoria
        levelFinished = true;
        // AQUÍ TERMINA EL JUEGO
    }
}

// Si toca el suelo lunar (que no es la plataforma)
if (player->y() > 700) {
    qDebug() << "¡CRASH! Te estrellaste contra el polvo lunar.";
    player->setPos(100, 100); // Reiniciar
}
}
}

```

levelbase.cpp

```
#include "levelbase.h"
```

```
LevelBase::LevelBase(QGraphicsScene *scene, Game* game)
```

```

{
    this->scene = scene;
    this->game = game;
}

```

```
LevelBase::~~LevelBase()
```

```
{
```

```
}
```

```
void LevelBase::clear()
```

```
{
```

```
    scene->clear();
```

```
}
```

main.cpp

```
#include "gamewindow.h"
```

```
#include <QApplication>
```

```
#include <QLocale>
```

```
#include <QTranslator>
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    QApplication a(argc, argv);
```

```
    QTranslator translator;
```

```
    const QStringList uiLanguages = QLocale::system().uiLanguages();
```

```
    for (const QString &locale : uiLanguages) {
```

```
        const QString baseName = "cronicas_del_tiempo_" + QLocale(locale).name();
```

```
        if (translator.load(":/i18n/" + baseName)) {
```

```
            a.installTranslator(&translator);
```

```
            break;
```

```
        }
```

```
    }
```

```
    GameWindow w;
```

```
w.show();  
return a.exec();  
}
```

playercaravan.cpp

```
#include "playercaravan.h"
```

```
PlayerCaravan::PlayerCaravan()
```

```
{  
    setPixmap(QPixmap(":/sprites/player_caravan.png").scaled(80, 80,  
Qt::KeepAspectRatio));  
    speed = 250.0;  
    up = down = left = right = false;  
    setFlag(QGraphicsItem::ItemIsFocusable);  
}
```

```
void PlayerCaravan::update(double dt)
```

```
{  
    vx = 0; vy = 0;  
    if (up) vy = -speed;  
    if (down) vy = speed;  
    if (left) vx = -speed;  
    if (right) vx = speed;
```

```
Entity::update(dt); // Mueve la posición
```

```
// Límites de pantalla
```

```
if (x() < 0) setPos(0, y());
```

```
if (x() > 1200) setPos(1200, y());
```

```

        if (y() < 0) setPos(x(), 0);
        if (y() > 640) setPos(x(), 640);
    }

void PlayerCaravan::keyPressEvent(QKeyEvent *event)
{
    if (event->key() == Qt::Key_W || event->key() == Qt::Key_Up) up = true;
    if (event->key() == Qt::Key_S || event->key() == Qt::Key_Down) down = true;
    if (event->key() == Qt::Key_A || event->key() == Qt::Key_Left) left = true;
    if (event->key() == Qt::Key_D || event->key() == Qt::Key_Right) right = true;
}

void PlayerCaravan::keyReleaseEvent(QKeyEvent *event)
{
    if (event->key() == Qt::Key_W || event->key() == Qt::Key_Up) up = false;
    if (event->key() == Qt::Key_S || event->key() == Qt::Key_Down) down = false;
    if (event->key() == Qt::Key_A || event->key() == Qt::Key_Left) left = false;
    if (event->key() == Qt::Key_D || event->key() == Qt::Key_Right) right = false;
}

```

playermaraton.cpp

```

#include "playermaraton.h"
#include <QDebug>

PlayerMaraton::PlayerMaraton()
{
    // 1. Cargar el Sprite del Corredor
    QPixmap sprite(":/sprites/player_maraton.png");
}

```

```

// Si la imagen es muy grande, la escalamos un poco (ajusta el 100, 100 si es necesario)
setPixmap(sprite.scaled(80, 80, Qt::KeepAspectRatio));

// 2. Configuración de Físicas
vx = 0;
vy = 0;
speed = 400.0;    // Velocidad al correr
jumpForce = -700.0; // Fuerza hacia ARRIBA (negativo es arriba en Qt)
gravity = 1800.0;  // Gravedad fuerte para que caiga rápido
groundY = 530;    // Altura del suelo (ajústalo según tu fondo)

movingLeft = false;
movingRight = false;
isJumping = false;

setFlag(QGraphicsItem::ItemIsFocusable);
}

void PlayerMaraton::update(double dt)
{
    // --- MOVIMIENTO HORIZONTAL ---
    if (movingRight) vx = speed;
    else if (movingLeft) vx = -speed;
    else vx = 0; // Frenar si no toca teclas

    // --- GRAVEDAD Y SALTO ---
    // Aplicar gravedad constantemente

```

```

vy += gravity * dt;

// --- MOVER ---
// La clase padre (Entity) mueve el objeto según vx y vy
Entity::update(dt);

// --- DETECCIÓN DE SUELO ---
// Si el personaje baja más allá del suelo, lo detenemos
if (y() >= groundY) {
    setPos(x(), groundY); // Corregir posición
    vy = 0;               // Detener caída
    isJumping = false;    // Ya no está saltando, puede saltar de nuevo
}

// Límites de pantalla (izquierda/derecha)
if (x() < 0) setPos(0, y());
if (x() > 1200) setPos(1200, y());
}

void PlayerMaraton::keyPressEvent(QKeyEvent *event)
{
    if (event->key() == Qt::Key_Left || event->key() == Qt::Key_A) {
        movingLeft = true;
    }
    if (event->key() == Qt::Key_Right || event->key() == Qt::Key_D) {
        movingRight = true;
    }
    // SALTO: Solo si no está saltando ya (para evitar doble salto infinito)

```

```
    if ((event->key() == Qt::Key_Space || event->key() == Qt::Key_Up || event->key() ==  
Qt::Key_W) && !isJumping) {  
        vy = jumpForce;  
        isJumping = true;  
    }  
}
```

```
void PlayerMaraton::keyReleaseEvent(QKeyEvent *event)  
{  
    if (event->key() == Qt::Key_Left || event->key() == Qt::Key_A) {  
        movingLeft = false;  
    }  
    if (event->key() == Qt::Key_Right || event->key() == Qt::Key_D) {  
        movingRight = false;  
    }  
}
```